

Relatório de Auditoria de Segurança — GameHub Platform

Revisão de Código-Fonte, Arquitetura e Modelagem de Ameaças

Data do Relatório: Setembro de 2026	Versão do Sistema: 0.1.0-prod
Escopo Auditado: Repositório Completo (app/, games/, static/, deploy)	Classificação: Confidencial / Uso Interno
Ambiente: FastAPI, WebSockets, SQLite, HTML5 Canvas	Status da Avaliação: 10 Achados Identificados

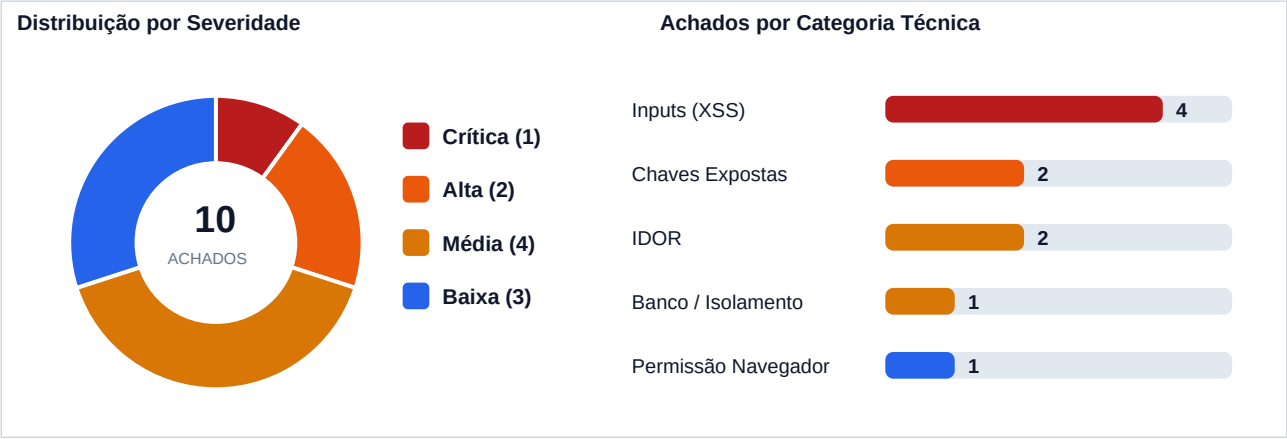
Nota Metodológica & Detecção da Stack Tecnológica

A auditoria cobriu o código-fonte através de análise estática e mapeamento de fluxo de dados. Como o **GameHub** adota uma arquitetura intencional de *plataforma casual no-login* (sem contas persistentes corporativas), as categorias clássicas foram adaptadas à realidade técnica do projeto:

1. Banco sem tranca	Mapeado para isolamento de sessões (UUIDs) no SQLite/Memória e integridade do dicionário global compartilhado (Word Bank).
2. Permissão no navegador	Mapeado para operações privilegiadas (endpoints de boletos, submissão de highscores e validação de jogadas em WebSocket).
3. IDOR	Verificação exaustiva de todas as rotas que recebem identificadores para manipulação de sessões de jogos (Snake, TD, Hex, Damas).
4. Chaves expostas	Inspeção de literais de senhas, defaults em variáveis de ambiente, configs Docker e varredura do histórico Git.
5. Inputs sem tratamento	Varredura de innerHTML no frontend (Vanilla JS) e inserção de dados de usuário (hints, scores, headers HTTP).

2. Resumo Executivo

A auditoria identificou **10 achados de segurança** distribuídos entre 1 Crítica, 2 Altas, 4 Médias e 3 Baixas. A principal superfície de risco reside em **injeções Stored XSS via manipulação direta de innerHTML** alimentadas por dados não sanitizados (cabeçalho X-Forwarded-For em logs, dicas de palavras cruzadas no banco e nomes em highscores). Em contrapartida, o projeto apresenta **sólida proteção contra SQL Injection** e **validação rigorosa de regras de jogo no backend**.



3. Pontos Fortes & Pontos Fracos

■■ PONTOS FORTES (Controles Verificados) • Zero SQL Injection: 100% das consultas utilizam SQLAlchemy com select(Model).where(...) parameterizado. Zero concatenação de strings SQL. • Validação de Jogadas no Servidor: O WebSocket em app/websocket.py valida regras de damas e palavras cruzadas no backend, impedindo trapaças por clientes adulterados. • Entropia de Sessão: IDs de jogos utilizam uuid.uuid4() (128 bits aleatórios), prevenindo adivinhação trivial de salas. • Autenticação em Módulos Sensíveis: O router boletos.py exige Depends(_authenticate) no servidor para todos os endpoints restritos.	■■ PONTOS FRACOS (Riscos Centrais) • Stored XSS em Vários Módulos: Uso de innerHTML sem sanitização em boletos.html (logs), crossword (dicas) e tabelas de recordes. • Poluição Global de Dicionário: POST /api/words permite que usuários anônimos insiram palavras/dicas no SQLite compartilhado. • Credencial Padrão sem Guard de Startup: BOLETOS_PASS cai em default change-me se não configurado no ambiente. • IDOR / Negação de Serviço em Snake: Endpoint DELETE /snake/{id} permite apagar partidas ativas sem autenticação de dono.
---	---

4. Tabela de Achados Detalhados por Categoria

Todos os achados foram confirmados diretamente no código-fonte do repositório.

Sev.	Categoria	Arquivo : Linha	Descrição & Impacto
CRÍTICA	Inputs (XSS)	app/boletos.py:168 static/boletos.html:407	Stored XSS via X-Forwarded-For: O cabeçalho HTTP é salvo em _logs sem filtro e renderizado em innerHTML no painel de logs. Permite roubo de token de sessão.
ALTA	Inputs (XSS)	app/main.py:138 crossword/board.js:117	Stored XSS via Dica de Palavra: POST /api/words insere dicas arbitrárias no SQLite global. Ao gerar palavras cruzadas, a dica é injetada diretamente no innerHTML.
ALTA	Chaves / Segredos	app/boletos.py:30-31	Credencial Padrão Previsível: Fallback BOLETOS_PASS = change-me ativo sem validação de startup em produção, permitindo acesso não autorizado.
MÉDIA	Banco / Isolamento	app/main.py:138-145	Poluição do Banco de Palavras: Endpoint público sem autenticação permite que qualquer cliente insira palavras inadequadas no dicionário global.
MÉDIA	IDOR / DoS	games/snake/routes.py:106	Deleção de Sessão Arbitrária: DELETE /snake/{id} finaliza a partida de qualquer usuário sem validação de posse de sessão.
MÉDIA	Inputs (XSS)	bomberman/game.js:2050 tower_defense/index.html:2553	Stored XSS em Recordes: Nomes enviados via API de highscores são inseridos via innerHTML sem sanitização nas tabelas de pontuação.
MÉDIA	Chaves / Segredos	Git commit 4da1006	Senha no Histórico Git: Credencial legada 123456secreta permanece gravada nos commits históricos do repositório.
BAIXA	Permissão / Integridade	bomberman/routes.py:86 tower_defense/routes.py:141	Falta de Game Proof em Scores: Endpoints aceitam qualquer pontuação sem prova de conclusão válida de partida.
BAIXA	IDOR / Enumeração	app/main.py:202-210	Consulta Irrestrita de Rating: GET /api/ratings/{player_id} expõe histórico de qualquer identificador sem restrição.
BAIXA	Inputs (XSS) / Defesa	app/main.py:72-79	Ausência de CSP e Headers de Proteção: Falta de Content-Security-Policy, X-Frame-Options e X-Content-Type-Options.

5. Recomendações Priorizadas (Roadmap de Correção)

P1 — Correção Imediata de XSS (innerHTML → textContent)

Substituir concatenações diretas de innerHTML por textContent ou criação segura de elementos DOM (document.createElement) em todos os painéis de logs, dicas de palavras cruzadas e murais de recordes.

P2 — Proteção e Sanitização do Endpoint /api/words

Exigir chave de API ou desabilitar o endpoint POST /api/words em ambientes públicos, além de sanitizar tags HTML e limitar o comprimento dos campos word e hint.

P3 — Validação de Startup para Segredos em Produção

Adicionar checagem no lifespan do FastAPI em app/main.py para lançar erro fatal se BOLETOS_PASS for igual a 'change-me' ou '123456secreta' fora de ambiente de desenvolvimento.

P4 — Adição de Cabeçalhos de Segurança HTTP (CSP, X-Frame-Options)

Configurar middleware HTTP adicionando Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; X-Frame-Options: DENY e X-Content-Type-Options: nosniff.

P5 — Purga de Segredos do Histórico Git

Utilizar ferramenta como git-filter-repo ou BFG Repo-Cleaner para expurgar commits legados contendo senhas literais caso o repositório seja tornado público.

6. Issues Prontas para o GitHub

Copie e cole diretamente os blocos abaixo para criar as issues no GitHub do projeto:

```
--- ISSUE 1 ---
[Segurança] Stored XSS no painel de logs de boletos via cabeçalho X-Forwarded-For

**Labels:** `security`, `severity: critical`, `bug`

#### Descrição
O cabeçalho HTTP `X-Forwarded-For` é extraído em `app/boletos.py:168-171` sem sanitização e armazenado na lista de logs da aplicação. Ao acessar a interface de visualização de logs (`static/boletos.html:407-410`), o valor do IP é interpolado diretamente em `innerHTML`, permitindo a execução de JavaScript arbitrário no contexto de um usuário autenticado.

#### Evidência
- `app/boletos.py:168`: `forwarded = request.headers.get("X-Forwarded-For")`
- `static/boletos.html:407`: `container.innerHTML = logs.map(l => '<span class="log-ip">${l.ip}</span>...').join('')`

#### Impacto
Execução remota de JavaScript no navegador do operador/usuário autenticado, possibilitando roubo de tokens Bearer e ações não autorizadas.

#### Sugestão de Correção
1. Validar formato IPv4/IPv6 no backend (`ipaddress.ip_address`).
2. Substituir `innerHTML` em `boletos.html` por manipulação segura com `document.createElement` e `textContent`.

#### Critérios de Aceite
- [ ] Payload `
```

```
--- ISSUE 2 ---
[Segurança] Stored XSS e poluição do banco de palavras cruzadas via POST /api/words

**Labels:** `security`, `severity: high`, `bug`

#### Descrição
O endpoint `POST /api/words` é público e permite inserção irrestrita de palavras e dicas no banco de dados compartilhado. As dicas (`hint`) são posteriormente renderizadas no frontend de Palavras Cruzadas (`games/crossword/static/board.js:117,131`) diretamente via `innerHTML`, criando um vetor de Stored XSS que afeta outros jogadores.

#### Evidência
- `app/main.py:138`: `POST /api/words` não exige autenticação ou moderação.
- `games/crossword/static/board.js:117`: `acrossList.innerHTML = across.map(clue => '<span class="clue-text">${clue.clue}</span>').join('')`

#### Impacto
Qualquer visitante pode infectar o dicionário global com payloads XSS persistentes que serão executados por qualquer jogador que abrir uma partida de Palavras Cruzadas.

#### Sugestão de Correção
1. Sanitizar tags HTML no backend com `html.escape()` antes de salvar ou escapar no frontend.
2. Trocar `innerHTML` por `textContent` no `board.js`.
3. Proteger `POST /api/words` com autenticação ou desabilitá-lo em produção.

#### Critérios de Aceite
- [ ] Dicas contendo tags HTML não disparam execução de scripts.
- [ ] `board.js` utiliza `textContent` para renderizar `clue.clue`.
--- FIM ISSUE 2 ---
```

```
--- ISSUE 3 ---
[Segurança] Credencial padrão fraca e falta de validação de inicialização em app/boletos.py

**Labels:** `security`, `severity: high`, `configuration`

#### Descrição
A configuração de autenticação em `app/boletos.py:30-31` utiliza o valor padrão `"change-me"` caso a variável de ambiente `BOLETOS_PASS` não seja informada. Em produção, se a variável for omitida, o sistema aceita credenciais padrão conhecidas publicamente.

#### Evidência
- `app/boletos.py:30-31`: `TEST_PASS = os.getenv("BOLETOS_PASS", "change-me")`
```

```
### Impacto
Acesso não autorizado ao gerador de PDFs e ao painel de logs por invasores usando credenciais default.

### Sugestão de Correção
Adicionar verificação no `lifespan` da aplicação que impede a inicialização em modo de produção se `BOLETOS_PASS`
for igual a `"change-me"` ou estiver ausente.

### Critérios de Aceite
- [ ] A aplicação recusa iniciar em produção sem `BOLETOS_PASS` forte e configurado.
--- FIM ISSUE 3 ---
```

```
--- ISSUE 4 ---
[Segurança] Stored XSS nos murais de recordes (Bomberman, Tower Defense, Colônia Hex)

**Labels:** `security`, `severity: medium`, `bug`

### Descrição
Os nomes dos jogadores submetidos nas tabelas de recordes (`POST /*/highscores`) são inseridos diretamente no DOM
através de `innerHTML` nos clientes web dos jogos Bomberman (`game.js:2050`), Tower Defense (`index.html:2553`) e
Colônia Hex (`index.html:830`).

### Evidência
- `games/bomberman/static/game.js:2050`: `listEl.innerHTML = scores.map(s => '<span`
class="rank">${s.name}</span>...').join('')`
- `games/tower_defense/static/index.html:2553`: `list.innerHTML = scores.map(s => '<span`
class="hs-name">${s.name}</span>...').join('')`

### Impacto
Execução de scripts XSS nos navegadores de todos os jogadores que visualizam os recordes.

### Sugestão de Correção
Escapar caracteres especiais de HTML (`<`, `>`, `&`, `"`, ``) nos nomes ao receber no backend e utilizar
`textContent` ao criar as linhas de score no frontend.

### Critérios de Aceite
- [ ] Nomes como `
```

Impacto

Aumento da superfície para ataques de Clickjacking (incorporação em iframes maliciosos) e menor mitigação em profundidade contra XSS.

Sugestão de Correção

Adicionar no middleware HTTP:

```
- `X-Frame-Options: DENY`  
- `X-Content-Type-Options: nosniff`  
- `Content-Security-Policy: default-src 'self'; script-src 'self' 'unsafe-inline'; style-src 'self' 'unsafe-inline' https://fonts.googleapis.com; font-src 'self' https://fonts.gstatic.com;`
```

Critérios de Aceite

```
- [ ] Todas as respostas HTTP incluem os cabeçalhos de segurança recomendados.  
--- FIM ISSUE 6 ---
```